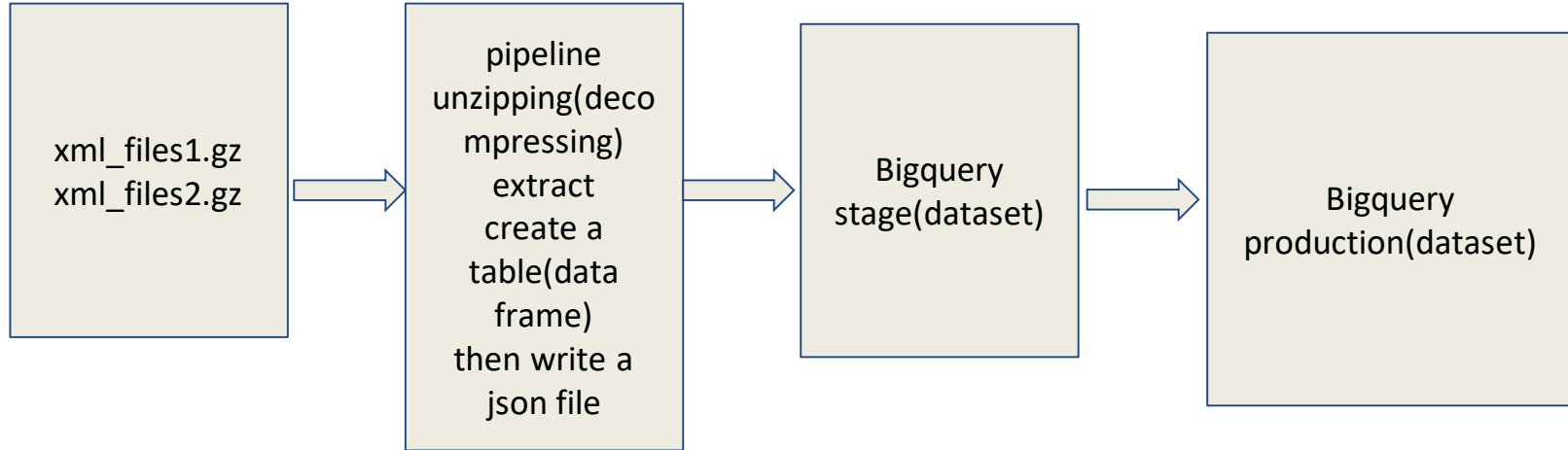


Real world example of group by



mesinfoid

node- 1

node -2

node -3

init time

23-08-2025 T012530

UTC

```
select max(mesinfo_id), max(init),.....,any_value(job_id) as  
job_id from table group by mesinfoid, init
```

insert into final table from
(select max(mesinfo_id), max(init),.....,any_value(job_id) as
job_id from table group by mesinfo_id, init,file_name)

mesinfo_id	init_time	job_id	process_id	file_name
node1	23-08-2025 T01253	23	123456	file1
node2	23-08-2025 T01253	23	123456	file1
node1	23-08-2025 T01215	23	123456	file1
node1	23-08-2025 T01253	23	123456	file2
node2	23-08-2025 T01253	23	123456	file2
node2	23-08-2025 T01253	23	123456	file2

Module

A module is a file containing Python definitions and statements.

A module is a file containing group of variables, methods, function and classes etc.

They are executed only the first time the module name is encountered in an import statement.

Assume that you are building a very large project, it will be very difficult to manage all logic within one single file so If you want to separate your similar logic to a separate file, you can use module.

Type of Modules:-

- User-defined Modules
- Built-in Modules

Ex:- array, math, numpy, sys,pandas, datetime

How to use Module

import statement is used to import modules.

Syntax:-

```
import module_name
```

```
import module_name as alias_name
```

```
from module_name import function_name1, function_name2,....., function_nameN
```

```
from module_name import f_name as alias_f_name
```

```
from module_name import *
```

from module_name import *

This imports all names except those beginning with an underscore (_).

Ex:- from cal import *

How to access Methods, Functions, Variable and Classes ?

You can access the functions directly by it's name.

Ex:-

add(10, 20)

sub(20, 10)

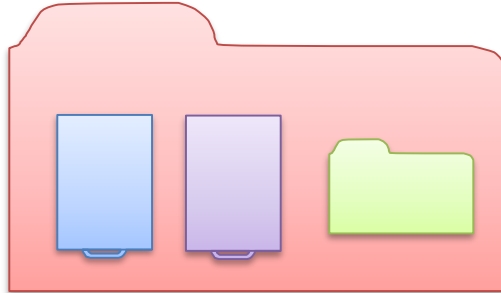
Package

Packages are a way of structuring Python's module namespace by using “dotted module names”.

A package can have one or more modules which means, a package is collection of modules and packages.

A package can contain packages.

Package is nothing but a Directory/Folder



Creating Package

Package is nothing but a Directory/Folder which MUST contain a special file called `__init__.py`.

`__init__.py` file can be empty, it indicates that the directory it contains is a Python package, so it can be imported the same way a module can be imported.

- SMS
 - Admin
 - dashboard.py
 - service.py
 - product.py
 - logout.py
 - __init__.py
 - User
 - profile.py
 - request.py
 - logout.py
 - __init__.py
 - Tech
 - profile.py
 - work.py
 - logout.py
 - __init__.py
- geekyshows.py
- __init__.py

How to use Package

Syntax:- `from packageName.moduleName import fun_name`

Syntax:- `from packageName.subPackageName.moduleName import fun_name`

Ex:- `from Admin.service import admin_service`

Ex:- `from Admin.Common.footer import admin_common_footer`

How to Access Variable, Function, Method, Class etc.

Syntax:- `functionName()`

Ex:-

`admin_service( )`

`admin_common_footer( )`

__all__

if a package's `__init__.py` code defines a list named `__all__`, it is taken to be the list of module names that should be imported when `from package import *` is encountered.

```
__all__ = ['dashboard', 'service', 'product']
```

Package Ex

Folder structure:

-pipeline

- transformer
 - ◆ nokia
- `__init__.py`
- `xml_reader.py`
- `xml_transformer.py`
 - ◆ samsung
 - ◆ huawei
 - ◆ `process_reader.py`
 - ◆ `transformer_factory.py`
- sources
- utils

`__init__.py`

```
from transform.nokia.xml_transformer import table
```

```
__all__ = [table]
```

`transformer_factory`

```
from transformer.nokia import table
```

or

```
from transformer.nokia.xml_transformer import table
```

Abstract Class

A class derived from **ABC** class which belongs to *abc* module, is known as abstract class in Python.

ABC Class is known as Meta Class which means a class that defines the behavior of other classes. So we can say, Meta Class ABC defines that the class which is derived from it becomes an abstract class.

Abstract Class can have abstract method and concrete methods.

Abstract Class needs to be extended and its method implemented.

PVM can not create objects of an abstract class.

Ex:-

```
from abc import ABC, abstractmethod
```

```
Class Father(ABC):
```

Rules:

- JVM can not create objects of an abstract class.
- It is not necessary to declare all methods abstract in a abstract class.
- Abstract Class can have abstract method and concrete methods.
- If there is any abstract method in a class, that class must be abstract.
- The abstract methods of an abstract class must be defined in its child class/subclass.
- If you are inheriting any abstract class that have abstract method, you must either provide the implementation of the method or make this class abstract.

Abstract Method

```
from abc import ABC, abstractmethod
```

```
class Father(ABC):  
    @abstractmethod  
  
    def disp(self):  
        print('disp class')  
  
    def show(self):  
        print("Concrete Method")
```

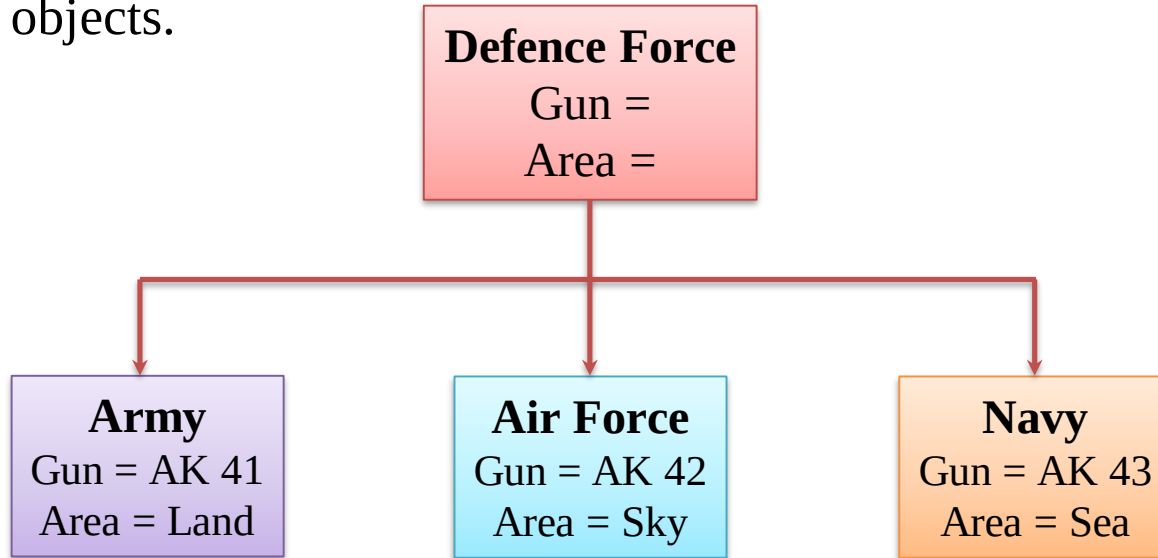
```
class Child(Father) :  
  
    def __init__(self):  
        self.name = 'Raju'
```

```
    def disp(self):  
        print('pass')
```

```
a = Child()  
print(a.disp())
```

When use Interface

We use interface when all the features need to be implemented differently for different objects.



Date and Time

Following modules are used to work with date, time, and duration.

- time
- datetime

Time Modules

- `time ( )` Function – This function return the time in seconds since the epoch as a floating point number. The specific date of the epoch and the handling of leap seconds is platform dependent.
- `ctime ( )` Function – This function is used to get current date and time. When we pass epoch time in seconds to the function, it returns corresponding date and time in string format. When we do not pass epoch time, it returns current date and time in string format.

datetime Module

```
import time
from datetime import datetime, timedelta

reminder_time = datetime.now() + timedelta(seconds=5)
print(f"Reminder set for {reminder_time.strftime('%H:%M:%S')}")

while datetime.now() < reminder_time:
    time.sleep(100)

print("Time to drink water!")
```

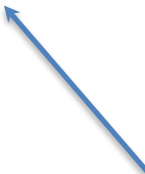
Formatting Date and Time

strftime() Method – This method is used to format the content of datetime, date and time class object. strftime represents string format to time. This method convert the object into a specified format and returns the formatted string.

Ex:-

```
dt = datetime.today()
```

```
newdt = dt.strftime("%B, %d, %Y")
```



Format Code

Format Code

Format Code	Meaning	Example
%a	Weekday in short name	Sun, Mon, ..., Sat
%A	Weekday in full name	Sunday, Monday, ..., Saturday
%d	Day of month with 0 padded	01, 02, ..., 30, 31
%b	Month in short Name	Jan, Feb,, Dec
%B	Month in full Name	January,, December
%m	Month in number with 0 padded	01, 02, ..., 12
%y	Year in short with 0 padded, without century	00, 01, 02, ..., 99
%Y	Year in Full with century	0001, 0002,, 9999

Format Code

Format Code	Meaning	Example
%H	Hours with 0 padded (24 hours clock)	00, 01, 02,....., 23
%I	Hours with 0 padded (12 hours clock)	01, 02,....., 12
%p	AM/PM	AM, PM
%M	Minute with 0 padded	00, 01,, 59
%S	Second with 0 padded	00, 01,, 59
%f	Microsecond with 0 padded	000000,....., 999999
%Z	Time zone name	(empty), UTC, CST, EST
%j	Day number of year with 0 padded	001, 002,....., 366
%U	Week number of the year, Sunday as the first day of week with 0 padded	00, 01,, 53

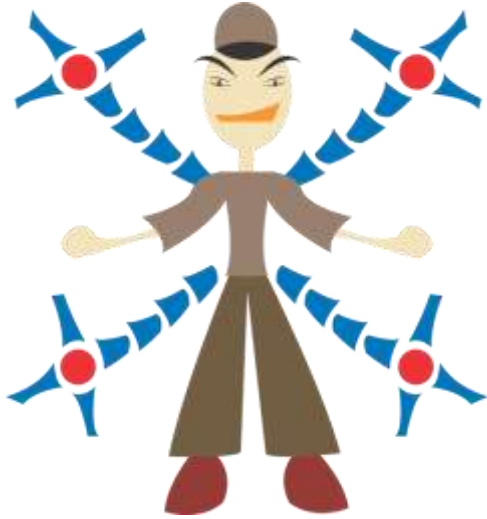
Format Code

Format Code	Meaning	Example
%c	Locale's appropriate date and time representation	Tue Jan 30 21:30:00 2019
%x	Locale's appropriate date representation	08/16/88 (None); 08/16/1988 (en_US); 16.08.1988 (de_DE)
%X	Locale's appropriate time representation	21:30:00 (en_US); 21:30:00 (de_DE)
%%	A literal '%' character	%

<https://docs.python.org/3.7/library/datetime.html#strftime-and-strptime-behavior>

Multitasking

Executing multiple task at the same time.



Type of Multitasking

- Process based Multitasking
- Thread based Multitasking

Process Based Multitasking

Executing multiple task at the same time where each task is a separate independent program(process), is called process based multitasking. It is suitable for Operating System level.

ENGS YTPython\Advance Python\Advance Python Code\14. Date and Time\7. CreatingTimedeltaClassObject.py - Notepad ++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

7. CreatingTimedeltaClassObject.py 8. CompareTwoDates.py 9. FormattingDT.py

```
1 # Creating Object of timedelta Class
2 from datetime import timedelta, date
3 td = timedelta(days=10)
4 d = date.today()
5 print(d+td)
6
```

Python file length: 132 lines: 6 Ln: 6 Col: 1 Sel: 0 | 0 Windows (CR LF) UTF-8 INS

Google x +

← → ↻ google.com ☆ 🔍 📧 📅 📌 📁 📁 📁 📁 📁 📁

Google

Google Search I'm Feeling Lucky

Calculator

Standard

0

MC MR M+ M- MS M*

% √ x² 1/x

CE C < ÷

7 8 9 ×

4 5 6 -

1 2 3 +

± 0 . =

Untitled - Notepad

File Edit Format View Help

Notepad

Thread Based Multitasking

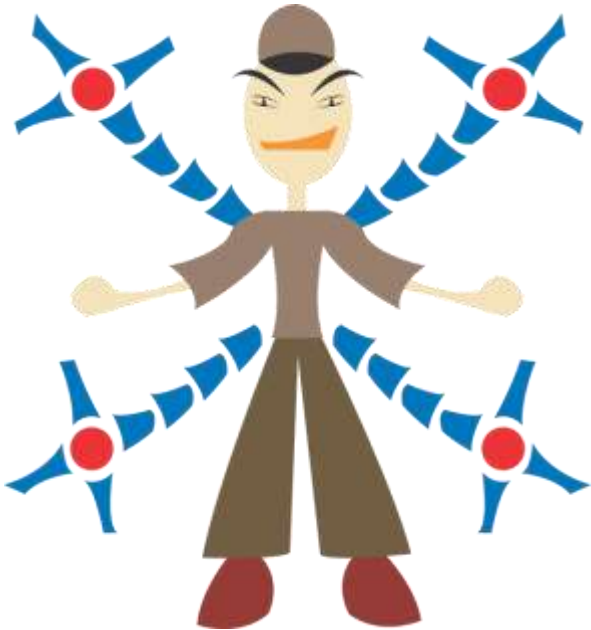
Executing multiple task at the same time where each task is a separate independent part of the same program(process), is called Thread based multitasking and each independent part is called Thread. It is suitable for Programmatic level.

Ex: - MS Word

Thread

Thread is a separate flow of execution. Every thread has a task.

- Flying Thread
- CallAuntyMay Thread
- Watching MJ Thread
- Doc Thread



Multithreading

Using Multiple Threads in program or process

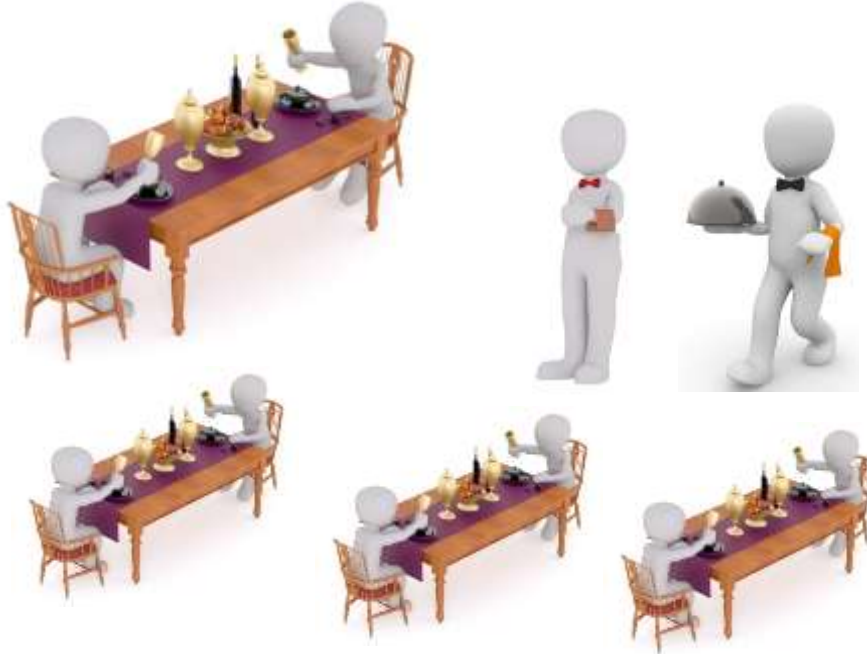
The main important application areas of multi threading are:

- Multimedia Graphic
- Animations
- Video Games
- Web Servers
- Application Servers



Multitasking using a Multiple Thread

When multiple tasks are executed at a time, then it is called Multi-tasking. For this purpose we need more than one thread and when we use more than one thread, it is called multi threading.



Main Thread

- When we start any Python Program, one thread begins running immediately, which is called Main Thread of that program created by PVM.
- The main thread is created automatically when your program is started.

```
import threading  
t = threading.current_thread().getName()  
print(t)
```

Thread Class's Methods

- `start ( )` – Once a thread is created it should be started by calling `start()` Method.
- `run( )` – Every thread will run this method when thread is started. We can override this method and write our own code as body of the method. A thread will terminate automatically when it comes out of the `run( )` Method.
- `join ( )` – This method is used to wait till the thread completely executes the `run ( )` method.

```
import threading
```

```
import time
```

```
def download_file(file_name, delay):
```

```
    print(f"📁 Starting download: {file_name}")
```

```
    time.sleep(delay) # simulate time taken to download
```

```
    print(f"✅ Finished download: {file_name}")
```

```
# Create threads
```

```
t1 = threading.Thread(target=download_file, args=("file1.mp4", 3))
```

```
t2 = threading.Thread(target=download_file, args=("file2.mp3", 2))
```

```
t3 = threading.Thread(target=download_file, args=("file3.pdf", 4))
```

```
# Start threads
```

```
t1.start()
```

```
t2.start()
```

```
t3.start()
```

```
# Wait for all to finish
```

```
t1.join()
```

```
t2.join()
```

```
t3.join()
```